

Rechnergestützte Numerik und Simulation

(am Beispiel von Matlab/Simulink)

Implementierung und Matlab-Praxis 2:

Funktionen und Operatoren

Inhalte

2. Funktionen und Operatoren

- 2.1 Eingebaute Funktionen
- 2.2 Matlab-Function
- 2.3 Suchliste
- 2.4 Besonderheiten

2. Funktionen – 2.1 Eingebaute Funktionen

Eingebaute Funktionen:

- Matlab enthält sehr viele Funktionen aus verschiedensten Gebieten, insbesondere durch die zahlreichen Toolboxes.
- Ein Funktionsaufruf besteht aus einem Namen und Übergabeparametern.
- Das Ergebnis kann einer Variablen oder einem Vektor von Variablen zugewiesen werden.
- Eine Übersicht über mathematische Standardfunktionen liefert der Befehl

```
>>help elfun
```



2. Funktionen – 2.1 Eingebaute Funktionen

Elementare Matrixfunktionen:

- `norm` Norm einer Matrix bzw. eines Vektors
- `rank` Rangberechnung
- `det` Determinante
- `trace` Spur
- `null` Kern der zugehörigen Abbildung

Lineare Gleichungssysteme

- `\` und `/` lösen von linearen Gleichungssystemen
- `inv` Matrixinvertierung
- `cond` Berechnung der Konditionszahl

Eigen- und Singulärwerte

- `eig` Bestimmung von Eigenwerten und Eigenvektoren
- `poly` Koeffizienten des charakteristischen Polynoms

Übersicht der Matrixfunktionen `>>help matlab/matfun`



2. Funktionen – 2.1 Eingebaute Funktionen

Achtung: Unterscheid elementweise Operationen und spezielle Matrix-Operation:

```
>> A = [1,2;4,3];  
>> sqrt(A)
```

```
ans =
```

```
1.0000    1.4142  
2.0000    1.7321
```

```
>> sqrtm(A)
```

```
ans =
```

```
0.7454 + 0.6667i    0.7454 - 0.3333i  
1.4907 - 0.6667i    1.4907 + 0.3333i
```

```
>> (sqrtm(A))^2
```

```
ans =
```

```
1.0000 + 0.0000i    2.0000 + 0.0000i  
4.0000 - 0.0000i    3.0000 - 0.0000i
```



2. Funktionen – 2.1 Eingebaute Funktionen

Siehe auch:

```
>> expm(A) ;  
>> logm(A) ;
```

2. Funktionen – 2.1 Eingebaute Funktionen

Trigonometrische Funktionen (Winkelangaben in Bogenmaß)

- `cos, sin, tan` Kosinus, Sinus, Tangens,
- `cot, sec, csc` Cotangens, Secans, Cosecans
- `acos, asin,` ...zugehörige inverse Funktionen
- `sinh, cosh,` ...hyperbolische Funktionen
- `asinh, acosh,` ...inverse hyperbolische Funktionen

Exponentialfunktionen

- `exp, pow2` Exponentialfunktion mit Basis `2` bzw. `2`
- `log, log10, log2` Logarithmus zur Basis `2`, `10` bzw. `2`
- `sqrt, realsqrt` Quadratwurzel bzw. Einschränkung auf `Reals`

Rechnen mit komplexen Zahlen

- `abs, angle, conj` Betrag, Argument (Winkel) und Konjugation
- `real, imag` Real- bzw. Imaginärteil

Sonstige

- `round, floor, ceil` runden, abrunden, aufrunden
- `mod, rem, sign` Modul, Divisionsrest, Vorzeichen
- `abs` Absolutwert



2. Funktionen – 2.1 Eingebaute Funktionen

Funktionen zur Datenanalyse:

- `max, min` Bestimmung maximaler bzw. minimaler Elemente
- `mean, median` Mittelwert bzw. Median
- `sort, sum` Aufsteigende Sortierung bzw. Summation von Elementen
- `cumsum, cumprod` kumulierte Summe bzw. kummuliertes Produkt

Differenzen:

- `diff` Differenzen aufeinander folgender Elemente
- `gradient` numerisch genäherter Gradient

Sonstige Funktionen:

- `conv, deconv` Multiplikation bzw. Division von Polynomen
- `fft, ifft` diskrete Fourier-Transformation und deren Inverse

Übersicht: `help matlab/datafun`



2. Funktionen – 2.1 Eingebaute Funktionen

Operatoren:

- Die rechte Seite einer Zuweisung darf ein beliebig komplizierter algebraischer Ausdruck sein.
- Konstanten und bekannte Variablen dürfen in sinnvoller Weise mit Operatoren verknüpft werden.
- Bei den arithmetischen Operatoren sind neben den üblichen (+, -, *, /) insbesondere das Potenzieren (^) und Transponieren (') von Bedeutung.
- Daneben gibt es auch die logischen Operatoren ~ (nicht), & (und), | (oder), xor() (exklusiv oder) und die relationalen Operatoren == (gleich), ~= (ungleich), <, ≤ (in Matlab <=), >, ≥ (in Matlab >=). Es gelten die üblichen Regeln für Punkt- vor Strichrechnungen. Andere Gruppierungen können mit runden Klammern () erreicht werden.
- Alle Operatoren sind auch als Funktionen verfügbar.
- Eine Übersicht über die Operatoren liefert der Befehl:

```
>> help ops
```



2. Funktionen – 2.1 Eingebaute Funktionen

Wichtig zum Finden von Funktionen:

- | | |
|-----------------|---|
| >> help ops | → Informationen zu elementaren Operatoren |
| >> help elemat | → Informationen zu Matrixoperationen |
| >> help elefun | → Informationen zu elementaren math. Funktionen |
| >> help specfun | → Informationen zu speziellen math. Operatoren |



2. Funktionen – 2.1 Eingebaute Funktionen

```
>> help ops
```

```
Operators and special characters.
```

```
Arithmetic operators.
```

plus	- Plus	+
uplus	- Unary plus	+
minus	- Minus	-
uminus	- Unary minus	-
mtimes	- Matrix multiply	*
times	- Array multiply	.*
mpower	- Matrix power	^
power	- Array power	.^
mldivide	- Backslash or left matrix divide	\
mrdivide	- Slash or right matrix divide	/
ldivide	- Left array divide	.\
rdivide	- Right array divide	./
kron	- Kronecker tensor product	kron

2. Funktionen – 2.1 Eingebaute Funktionen

Relational operators.

eq	- Equal	==
ne	- Not equal	~=
lt	- Less than	<
gt	- Greater than	>
le	- Less than or equal	<=
ge	- Greater than or equal	>=

Logical operators.

relop	- Short-circuit logical AND	&&
relop	- Short-circuit logical OR	
and	- Element-wise logical AND	&
or	- Element-wise logical OR	
not	- Logical NOT	~
xor	- Logical EXCLUSIVE OR	
any	- True if any element of vector is nonzero	
all	- True if all elements of vector are nonzero	

2. Funktionen – 2.1 Eingebaute Funktionen

Special characters.

colon	- Colon	:
paren	- Parentheses and subscripting	()
paren	- Brackets	[]
paren	- Braces and subscripting	{ }
punct	- Function handle creation	@
punct	- Decimal point	.
punct	- Structure field access	.
punct	- Parent directory	..
punct	- Continuation	...
punct	- Separator	,
punct	- Semicolon	;
punct	- Comment	%
punct	- Invoke operating system command	!
punct	- Assignment	=
punct	- Quote	'
transpose	- Transpose	.'
ctranspose	- Complex conjugate transpose	'
horzcat	- Horizontal concatenation	[,]
vertcat	- Vertical concatenation	[;]
subsasgn	- Subscripted assignment	(), { }, ..
subsref	- Subscripted reference	(), { }, ..
subsindex	- Subscript index	

2. Funktionen – 2.1 Eingebaute Funktionen

Bitwise operators.

- `bitand` - Bit-wise AND.
- `bitcmp` - Complement bits.
- `bitor` - Bit-wise OR.
- `bitmax` - Maximum floating point integer.
- `bitxor` - Bit-wise XOR.
- `bitset` - Set bit.
- `bitget` - Get bit.
- `bitshift` - Bit-wise shift.

Set operators.

- `union` - Set union.
- `unique` - Set unique.
- `intersect` - Set intersection.
- `setdiff` - Set difference.
- `setxor` - Set exclusive-or.
- `ismember` - True for set member.

2. Funktionen – 2.1 Eingebaute Funktionen

```
>>help elmat
```

```
Elementary matrices and matrix manipulation.
```

```
Elementary matrices.
```

zeros	- Zeros array.
ones	- Ones array.
eye	- Identity matrix.
repmat	- Replicate and tile array.
rand	- Uniformly distributed random numbers.
randn	- Normally distributed random numbers.
linspace	- Linearly spaced vector.
logspace	- Logarithmically spaced vector.
freqspace	- Frequency spacing for frequency response.
meshgrid	- X and Y arrays for 3-D plots.
accumarray	- Construct an array with accumulation.
:	- Regularly spaced vector and index into matrix.

2. Funktionen – 2.1 Eingebaute Funktionen

Basic array information.

- size - Size of array.
- length - Length of vector.
- ndims - Number of dimensions.
- numel - Number of elements.
- disp - Display matrix or text.
- isempty - True for empty array.
- isequal - True if arrays are numerically equal.
- isequalwithnans - True if arrays are numerically equal.

Matrix manipulation.

- cat - Concatenate arrays.
- reshape - Change size.
- diag - Diagonal matrices and diagonals of matrix.
- blkdiag - Block diagonal concatenation.
- tril - Extract lower triangular part.
- triu - Extract upper triangular part.
- fliplr - Flip matrix in left/right direction.
- flipud - Flip matrix in up/down direction.
- flipdim - Flip matrix along specified dimension.
- rot90 - Rotate matrix 90 degrees.
- :
- find - Find indices of nonzero elements.
- end - Last index.
- sub2ind - Linear index from multiple subscripts.
- ind2sub - Multiple subscripts from linear index.

2. Funktionen – 2.1 Eingebaute Funktionen

<code>isnan</code>	- True for Not-a-Number.
<code>isinf</code>	- True for infinite elements.
<code>isfinite</code>	- True for finite elements.
<code>j</code>	- Imaginary unit.
<code>why</code>	- Succinct answer.

Multi-dimensional array functions.

<code>ndgrid</code>	- Generate arrays for N-D functions and interpolation.
<code>permute</code>	- Permute array dimensions.
<code>ipermute</code>	- Inverse permute array dimensions.
<code>shiftdim</code>	- Shift dimensions.
<code>circshift</code>	- Shift array circularly.
<code>squeeze</code>	- Remove singleton dimensions.

Array utility functions.

<code>isscalar</code>	- True for scalar.
<code>isvector</code>	- True for vector.

2. Funktionen – 2.1 Eingebaute Funktionen

Specialized matrices.

<code>compan</code>	- Companion matrix.
<code>gallery</code>	- Higham test matrices.
<code>hadamard</code>	- Hadamard matrix.
<code>hankel</code>	- Hankel matrix.
<code>hilb</code>	- Hilbert matrix.
<code>invhilb</code>	- Inverse Hilbert matrix.
<code>magic</code>	- Magic square.
<code>pascal</code>	- Pascal matrix.
<code>rosser</code>	- Classic symmetric eigenvalue test problem.
<code>toeplitz</code>	- Toeplitz matrix.
<code>vander</code>	- Vandermonde matrix.
<code>wilkinson</code>	- Wilkinson's eigenvalue test matrix.

2. Funktionen – 2.1 Eingebaute Funktionen

Special variables and constants.

ans	- Most recent answer.
eps	- Floating point relative accuracy.
realmax	- Largest positive floating point number.
realmin	- Smallest positive floating point number.
pi	- 3.1415926535897....
i	- Imaginary unit.
inf	- Infinity.
nan	- Not-a-Number.

2. Funktionen – 2.2 M-Function

Matlab-Funktionen:

Es ist in Matlab auch möglich eigene Funktionen zu schreiben. Funktion

- werden durch das Schlüsselwort `function` gekennzeichnet,
- haben ihren eigenen Variablen-Bereich (Speicher),
- werden ‚call by value‘ aufgerufen,
- enden mit `end`.

Bsp.:

Datei: myfun.m

```
function [ output1, output2 ] = myfun( input1, input2 )  
  
output1 = input1 * input2;  
output2 = input1 + input2;  
  
end
```

```
>> [x, y] = myfun(2, 1)
```

```
x =  
    2  
  
y =  
    3
```



2. Funktionen – 2.2 M-Function

Der Return-Befehl:

- `return` → bewirkt den Abbruch der Funktionsverarbeitung und den Rücksprung zur aufrufenden Funktionen
- Bsp.:

```
function d = det(A)
    if isempty(A)
        d = 1;
        return
    else
        ...
    end
```



2. Funktionen – 2.2 M-Function

Matlab-Funktionen – weitere Eigenschaften:

- Funktionen die als allgemeine Resource zur Verfügung stehen sollen müssen in einer eigenen Datei `<funktionenname>.m` stehen die im aktuellen Suchpfad abgelegt ist. Das Schlüsselwort `function` muss in der ersten Zeile stehen.

Achtung: Hier ist der Dateiname entscheidend, nicht der Funktionskopf!!!

- Funktionen die nur in einem m-Script oder einer anderen m-Funktion verwendet werden sollen (Unterfunktionen – Sub-Functions) können am Ende des m-Scripts oder der m-Funktion (hinter dem `end` des Funktionsrumpfes) beschrieben werden. Diese Funktionen sind dann nicht von der Kommandozeile oder anderen Funktionen aufrufbar.

Achtung: Hier der Name im Funktionskopf entscheidend !!!



2. Funktionen – 2.2 M-Function

Bsp:

```
function [ output1, output2 ] = myfun( input1, input2 )  
  
output1 = input1 * input2;  
output2 = input1 + mysubfun(input2);  
  
end  
  
function [ output ] = mysubfun( input, input2 )  
  
output = 2 * input;  
  
end
```

2. Funktionen – 2.2 M-Function

Rekursivität:

M-Funktionen können direkt oder indirekt rekursiv ausgeführt werden.

Bsp.:

```
function out = myfun(in)

...

    out = x * myfun(y);

...

end
```



2. Funktionen – 2.2 M-Function

Verschachtelte Funktionen:

- Verschachtelte Funktionen sind Funktionen die innerhalb des Funktionsrumpfes einer anderen Funktion beschrieben werden.

Besonderheit: Hierbei ist der Variablenbereich aller verschachtelten Funktionen für die Hauptfunktion und umgekehrt sichtbar.



2. Funktionen – 2.3 Suchliste

Matlab-Arbeitsverzeichnis und Suchpfade:

Matlab findet alle m-Skripte (und m-Funktionen) die

- im aktuellen „Arbeitsverzeichnis“ oder
- in einem der Matlab „Suchpfade“

liegen.

Das Arbeitsverzeichnis kann über das Menü oder über Kommandozeilenbefehle gewechselt werden.



2. Funktionen – 2.3 Suchliste

Wichtige Befehle:

- `pwd` → Anzeigen des aktuellen Arbeitsverzeichnis
- `cd` → Wechseln des aktuellen Arbeitsverzeichnis
(absolute oder relative Pfadangabe)
- `dir` → Ordnerinhalt anzeigen
- `what` → Matlab-spezifische Dateien anzeigen
- `mkdir` → Erzeugen (make) eines Unterverzeichnisses
- `rmdir` → Löschen (remove) eines Unterverzeichnisses

(Viele Befehle sind an die DOS Befehle angelehnt)



2. Funktionen – 2.3 Suchliste

```
>> content = dir
```

```
content =
```

```
16x1 struct array with fields:
```

```
    name
```

```
    date
```

```
    bytes
```

```
    isdir
```

```
    datenum
```

```
>> content(3)
```

```
ans =
```

```
    name: 'firstmscript.m'
```

```
    date: '08-Okt-2010 13:35:38'
```

```
    bytes: 272
```

```
    isdir: 0
```

```
    datenum: 7.3442e+005
```



2. Funktionen – 2.3 Suchliste

```
>> dir
```

.	html	mscript5.m	mscript8.m
..	mscript3.m	mscript6.asv	mscript9.asv
firstmscript.asv	mscript4.asv	mscript6.m	mscript9.m
firstmscript.m	mscript4.m	mscript7.m	secondmscript.m

```
>> currentdir = pwd
```

```
currentdir =
```

```
C:\Users\Schulte\HS_OWL\Lehre\MatlabSeminar\Matlab\Kap3
```



2. Funktionen – 2.3 Suchliste

```
>> what
```

```
M-files in the current directory
```

```
C:\Users\Schulte\HS_OWL\Lehre\MatlabSeminar\Matlab\Kap3
```

```
firstmscript      mscript4          mscript6          mscript8
secondmscript
mscript3          mscript5          mscript7          mscript9
```

```
>> rer = what
```

```
rer =
```

```
    path: 'C:\Users\Schulte\HS_OWL\Lehre\MatlabSeminar\Matlab\Kap3'
         m: {9x1 cell}
        mat: {0x1 cell}
        mex: {0x1 cell}
        mdl: {0x1 cell}
         p: {0x1 cell}
   classes: {0x1 cell}
 packages: {0x1 cell}
```



2. Funktionen – 2.3 Suchliste

```
>> rer.m
```

```
ans =
```

```
'firstmscript.m'  
'mscript3.m'  
'mscript4.m'  
'mscript5.m'  
'mscript6.m'  
'mscript7.m'  
'mscript8.m'  
'mscript9.m'  
'secondmscript.m'
```

2. Funktionen – 2.3 Suchliste

Matlab-Suchpfade:

Die Matlab-Suchpfade werden mit dem `path`-Befehl angezeigt:

```
>> path
```

```
MATLABPATH
```

```
C:\Users\Schulte\Documents\MATLAB  
C:\Program Files\MATLAB\R2009b\toolbox\matlab\general  
C:\Program Files\MATLAB\R2009b\toolbox\matlab\ops  
C:\Program Files\MATLAB\R2009b\toolbox\matlab\lang  
C:\Program Files\MATLAB\R2009b\toolbox\matlab\elmat  
C:\Program Files\MATLAB\R2009b\toolbox\matlab\randfun  
C:\Program Files\MATLAB\R2009b\toolbox\matlab\elfun  
C:\Program Files\MATLAB\R2009b\toolbox\matlab\specfun  
C:\Program Files\MATLAB\R2009b\toolbox\matlab\matfun  
C:\Program Files\MATLAB\R2009b\toolbox\matlab\datafun  
C:\Program Files\MATLAB\R2009b\toolbox\matlab\polyfun  
C:\Program Files\MATLAB\R2009b\toolbox\matlab\funfun  
C:\Program Files\MATLAB\R2009b\toolbox\matlab\sparsfun
```

```
. . .
```



2. Funktionen – 2.3 Suchliste

Setzen von Matlab-Suchpfaden:

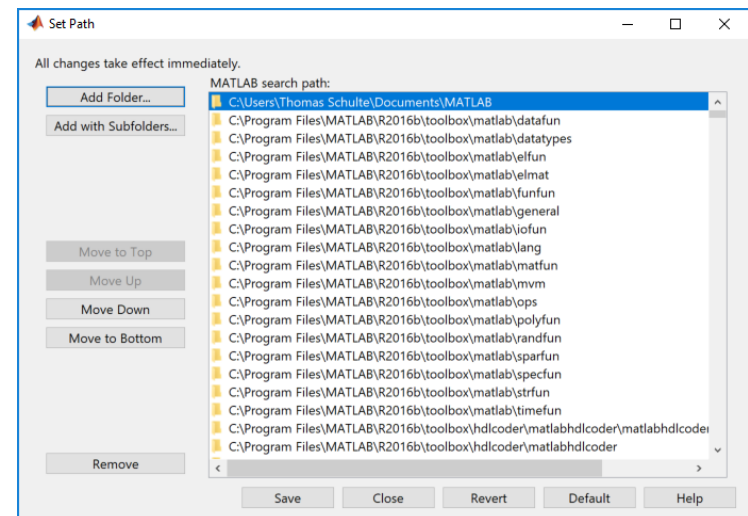
`addpath` → Setzen eines neuen Matlab-Suchpfaden

`rmpath` → Löschen eines Matlab-Suchpfaden

So erzeugte Änderungen der Suchpfadliste sind nur temporär!!!

`savepath` → Speichern der Suchpfade in `pathdef.m`, d.h. die Suchpfade werden beim Start von Matlab in Folge immer gesetzt.

`pathtool` → Öffnen des Suchpfade-Editors:



2. Funktionen – 2.3 Suchliste

Bsp.:

```
>> pwd
```

```
ans =
```

```
C:\Users\Schulte\HS_OWL\Lehre\MatlabSeminar\Matlab\Kap3
```

```
>> addpath(pwd)
```

```
>> path
```

```
MATLABPATH
```

```
C:\Users\Schulte\HS_OWL\Lehre\MatlabSeminar\Matlab\Kap3
```

```
C:\Users\Schulte\Documents\MATLAB
```

```
C:\Program Files\MATLAB\R2009b\toolbox\matlab\general
```

```
C:\Program Files\MATLAB\R2009b\toolbox\matlab\ops
```

```
C:\Program Files\MATLAB\R2009b\toolbox\matlab\lang
```

```
C:\Program Files\MATLAB\R2009b\toolbox\matlab\elmat
```



2. Funktionen – 2.3 Suchliste

Die Zugriffsreihenfolge bei mehreren Instanzen (für m-Skripte und m-Funktionen):

Bei mehreren gleichnamigen Skripten oder Funktionen in den Suchpfaden vorhanden (mehrere Instanzen) wird folgendermaßen priorisiert:

1. Arbeitverzeichnis
2. Suchpfade in der Reihenfolge der Suchpfadliste

```
>> pwd
```

```
ans =
```

```
C:\Users\Schulte\HS_OWL\Lehre\MatlabSeminar\Matlab\Kap3
```

```
>> which myfun
```

```
C:\Users\Schulte\HS_OWL\Lehre\MatlabSeminar\Matlab\Kap3\myfun.m
```

```
>> which myfun -all
```

```
C:\Users\Schulte\HS_OWL\Lehre\MatlabSeminar\Matlab\Kap3\myfun.m
```

```
C:\Users\Schulte\HS_OWL\Lehre\MatlabSeminar\Matlab\Kap3\subfolder\myfun.m %
```

```
Shadowed
```



2. Funktionen – 2.3 Suchliste

Die Zugriffsreihenfolge bei mehreren Instanzen (erweitert)

1. Variable im Arbeitsbereich der Funktion
2. Verschachtelte Funktion der aufrufenden Funktion
3. Unterfunktion der aufrufenden Funktion
4. Arbeitsverzeichnis
5. Suchpfade



2. Funktionen – 2.3 Suchliste

Einige weitere wichtige Befehle für Funktionen:

`exist` → prüft die Existenz einer Funktion

`rehash` → aktualisiert die Funktion im Speicher

Hinweis: *Normalerweise* werden Funktionen bei Änderungen automatisch aktualisiert. Vorsicht bei Verwendung anderer Editoren oder beim Kopieren von m-Dateien.

`clear` → löscht eine Funktion aus dem Speicher

2. Funktionen – 2.4 Besonderheiten

Globale Variablen:

- `global <variablenname1> <variablenname2> ...`
deklariert Variablen als global, d. h. als sichtbar für alle Funktionen und Skripte.
Die Variablen müssen aber in allen zugreifenden Files als global deklariert werden.
- `persistent <variablenname1> <variablenname2> ...`
deklariert Variablen als statisch, d. h. als sichtbar für mehrere Instanzen einer Funktion, solange die Funktion im Matlab-Speicher ist.



2. Funktionen – 2.4 Besonderheiten

Hilfe-Text bei eigenen Funktionen:

Es ist möglich Hilfetexte für eigene Funktionen im Matlab-Fenster anzeigen zu lassen. Hierzu wird der Hilfetext hinter der

```
function [ output1, output2 ] = myfun( input1, input2 )  
% MYFUN  
%   Calcultes something without any particular purpose.  
%  
%   Syntax: [x,y] = myfun(a,b)  
%  
%           where a and b are any numerical inputs.  
%  
%   See also myfun2  
  
output1 =  input1 * input2;  
output2 =  input1 + input2;  
  
end
```

2. Funktionen – 2.4 Besonderheiten

```
>> help myfun
```

```
MYFUN
```

```
Calcultes something without any particular purpose.
```

```
Syntax: [x,y] = myfun(a,b)
```

```
where a and b are any numerical inputs.
```

```
See also myfun2
```



2. Funktionen – 2.4 Besonderheiten

Funktionen mit Variablen Ein- und Ausgangsargumenten:

Es ist möglich m-Funktionen mit einer variablen Anzahl und variablen Typ von Ein- und Ausgangsvariablen zu versehen. Dies ist ein sehr mächtiges Mittel beim erstellen effizienter Bibliotheken:

`nargin` → Bestimmung der Anzahl der Eingangsgrößen

`nargout` → Bestimmung der Anzahl der Ausgangsgrößen

`varargin` → CellArray für variable Eingangsgröße

`varargout` → CellArray für variable Ausgangsgröße



2. Funktionen – 2.4 Besonderheiten

Bsp. 1:

```
function [output] = myfun( input1, input2, input3)

switch nargin
    case 1
        output = input1;
    case 2
        output = input1 + input2;
    case 3
        output = input1 + input2 + input3;
end
end
```

```
>> myfun3(1,2, 3)
```

```
ans =
     6
```

```
>> myfun3(1,2)
```

```
ans =
     3
```



2. Funktionen – 2.4 Besonderheiten

Bsp. 2:

```
function output = myfun(varargin)

parameter = varargin(1:2:end);
argument = varargin(2:2:end);

par2 = 0;
par3 = 0;
par4 = 0;

for par_index = 1 : length(parameter)
    switch parameter{par_index}
        case 'Par1'
            par1 = argument{par_index};
        case 'Par2'
            par2 = argument{par_index};
        case 'Par3'
            par3 = argument{par_index};
        case 'Par4'
            par4 = argument{par_index};
    end
end
output = par1 + par2^2 + par3^3 + par4^4;
end
```



2. Funktionen – 2.4 Besonderheiten

```
>> myfun4('Par1',2,'Par3',1)
```

```
ans =
```

```
3
```

```
>> myfun4('Par3',2,'Par1',1)
```

```
ans =
```

```
9
```



2. Funktionen – 2.4 Besonderheiten

Die ‚Evaluate‘-Befehle:

Die Gruppe der ‚Evaluate‘-Befehle (`eval`, `evalin`) bewirkt die Ausführung eines Strings als Matlab-Befehl.

Bsp.:

```
>> command = 'A = linspace(1,3,3)';  
>> eval(command)
```

A =

1 2 3

Der Befehl `eval` ist hilfreich um Anweisungen dynamisch (d.h. zur Laufzeit) aufzubauen.

Bsp.:

2. Funktionen – 2.4 Besonderheiten

Der Befehl `evalin` führt eine als String übergebene Anweisung auf dem spezifizierten Workspace aus. D. h. man kann aus einer m-Funktion heraus z. B. Befehle direkt auf dem Basis-Workspace ausführen.

Bsp.:

```
function myfun(input)

command = ['a = 2*',num2str(input),';'];
evalin('base',command);

end
```

```
>> myfun5(5)
```

```
>> a
```

```
a =
```

```
10
```

2. Funktionen – 2.4 Besonderheiten

Anstelle des Schlüsselwortes ‚base‘ kann bei

```
evalin(<ws>,<command_string>)
```

Auch ‚caller‘ als <ws> verwendet werden.

‚base‘ → Basis-Workspace
‚caller‘ → aufrufende Funktion.

Bei einfachen Variablenzuweisung für einen „fremden“ Workspace kann alternativ der Befehl `assign` verwendet werden.

```
...  
assignin('base','a',3);  
...
```

2. Funktionen – 2.4 Besonderheiten

Die Anweisung `inline` erzeugt eine Inline-Funktion aus einem String:

```
>> g = inline('x^2+x+1');  
>> g(3)
```

```
ans =
```

```
13
```

oder

```
>> g = inline('x^2+x+1+y');  
>> g(3,6)
```

```
ans =
```

```
19
```


2. Funktionen – 2.4 Besonderheiten

Ein und Ausschalten von Warnungen und Bildschirmausgaben:

```
>> echo on
```

→ Einschalten der Bildschirmausgabe Anweisung auch wenn diese mit “;” abgeschlossen sind.

```
>> echo off
```

→ Ausschalten der Bildschirmausgabe

```
>> warning off
```

→ Ausschalten der Ausgabe von Warnungen

```
>> warning('off', <ID>);
```

→ Ausschalten der Ausgabe von Warnungen mit dem ID-String <ID>

```
>> warning on
```

→ Einschalten der Ausgabe von Warnungen